BIP 360: Preparing Bitcoin for the Post-Quantum Era

By: Sadra Setarehdan, Rafik Hamzi, Arshiya Mahmoodinezhad

Date: April 6th 2026

Overview

01

Background and The Quantum Problem

02

Taproot (P2TR)

03

BIP 360: P2MR

04

Conclusion

A Brief Overview On Bitcoin

- Two mathematically linked keys, public and private
- Sign transactions with private
- Anyone can verify with public
- Can NOT be easily reversed (more on this later)

We know this from class, this is all standard

The Quantum Threat

Classical vs Quantum Computing

- Classical: bits, 0 or 1, one at a time
- Quantum: qubits, superposition of states
- Most tasks: no advantage
- Public-key math: massive advantage

Shor's Algorithm

- Can derive private key from public key
- Breaks elliptic curve cryptography
- Previously estimated: millions of qubits needed

Google Quantum AI

- 20× fewer resources than prior estimates
- < 500K physical qubits
- ~ 1,200 logical qubits
- Could crack a key in ~9 minutes

The Scale of exposure

6.9 M BTC

≈ ⅓ of the total supply

With exposed public keys on-chain

Who is Exposed?

- Early P2PK coins (public key in the clear)
- Satoshi's ~ 1 M BTC
- Unspent Taproot key-path addresses (more on this later)
- Any reused address after spending

P2PK and P2PKH

P2PK - Pay to Public Key

- Full public key stored directly on-chain
- Spender provides only a signature
- Public key permanently visible from the moment coins arrive
- All of Satoshi Nakamoto's coins use this format

P2PKH - Pay to Public Key Hash

- Only a hash of the public key on-chain
- Public key revealed only when spending
- Use once, sweep full balance ⇒ key only briefly exposed

P2SH and SegWit

P2SH - Pay to Script Hash

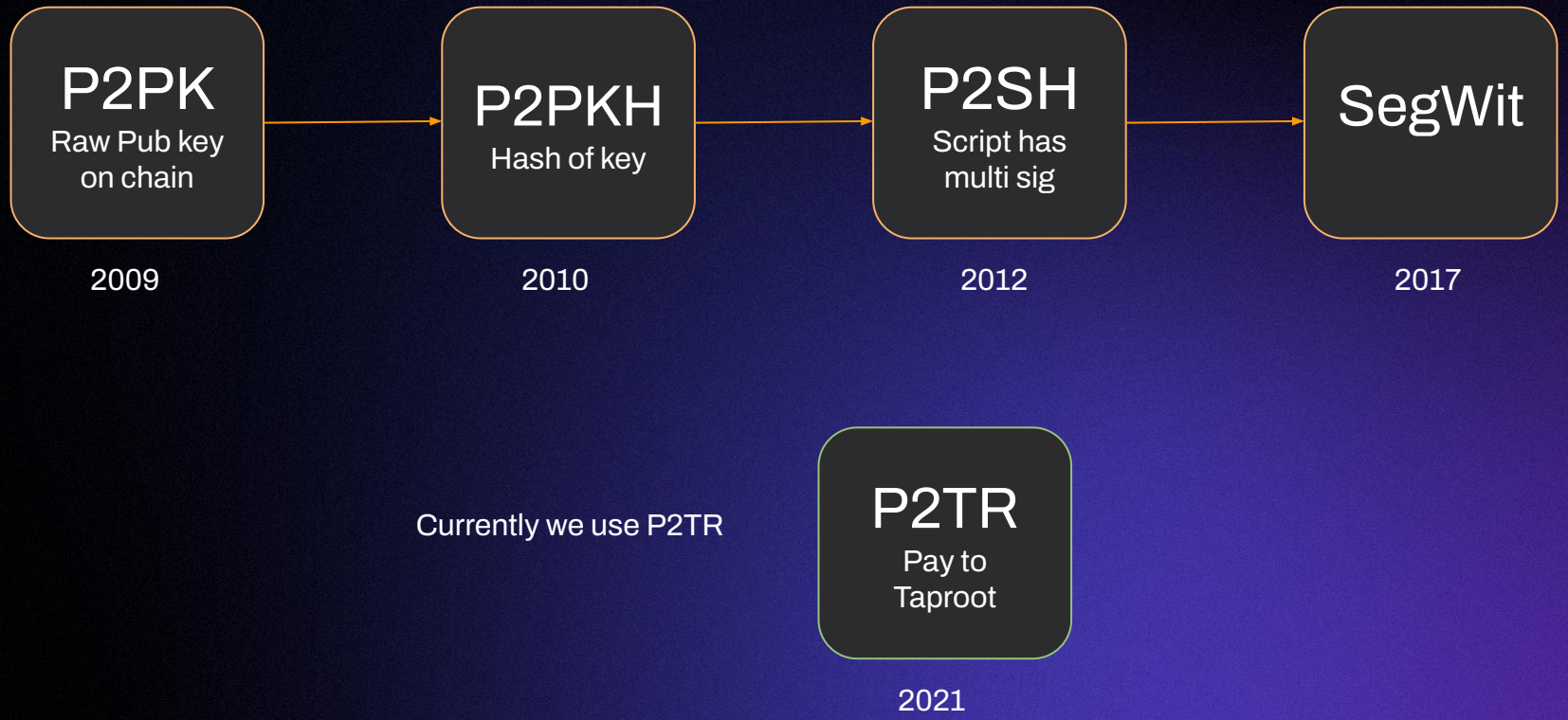
- Lock coins to the hash of an arbitrary script
- Enables multi-sig, time-locks, etc.
- Same security property: only a hash
- on-chain until spending

SegWit - Segregated Witness

- Moved signatures into a separate structure
- New versions: P2WPKH, P2WSH

All of these share the fact public keys stay hidden behind hashes until spend

Address Format Timeline



Pay to Tap Root (P2TR)

- Bitcoin's most significant upgrade in years
- New address format: P2TR (Pay-to-Taproot)
- Two spending paths from one address:

Key Path Spend

- Fast, cheap, default option
- Sign with private key
- Looks like any simple payment
- Complex conditions stay invisible

Script Path Spend

- Merkle tree of scripts
- Multi-sig, time-locks, recovery...
- Reveal only the branch you use
- Everything else stays hidden

Structure of a P2TR ScriptPubKey

Step 1: creating the merkle tree

The script tree contains all the custom scripts you'd like to be able to use for unlocking the P2TR output . You can use as many custom scripts as you want*, and you can organize the tree in any way you like.

The script tree and its merkle root is constructed in 3 steps:

- Leaves
- Organization
- Merkle Root

Step 2: creating the tweaked public key

After you've selected your public key and calculated the merkle root for your script tree, you now combine them to create a new tweaked public key (taproot) to place in the final P2TR locking script.

This tweaked public key is calculated in 2 steps:

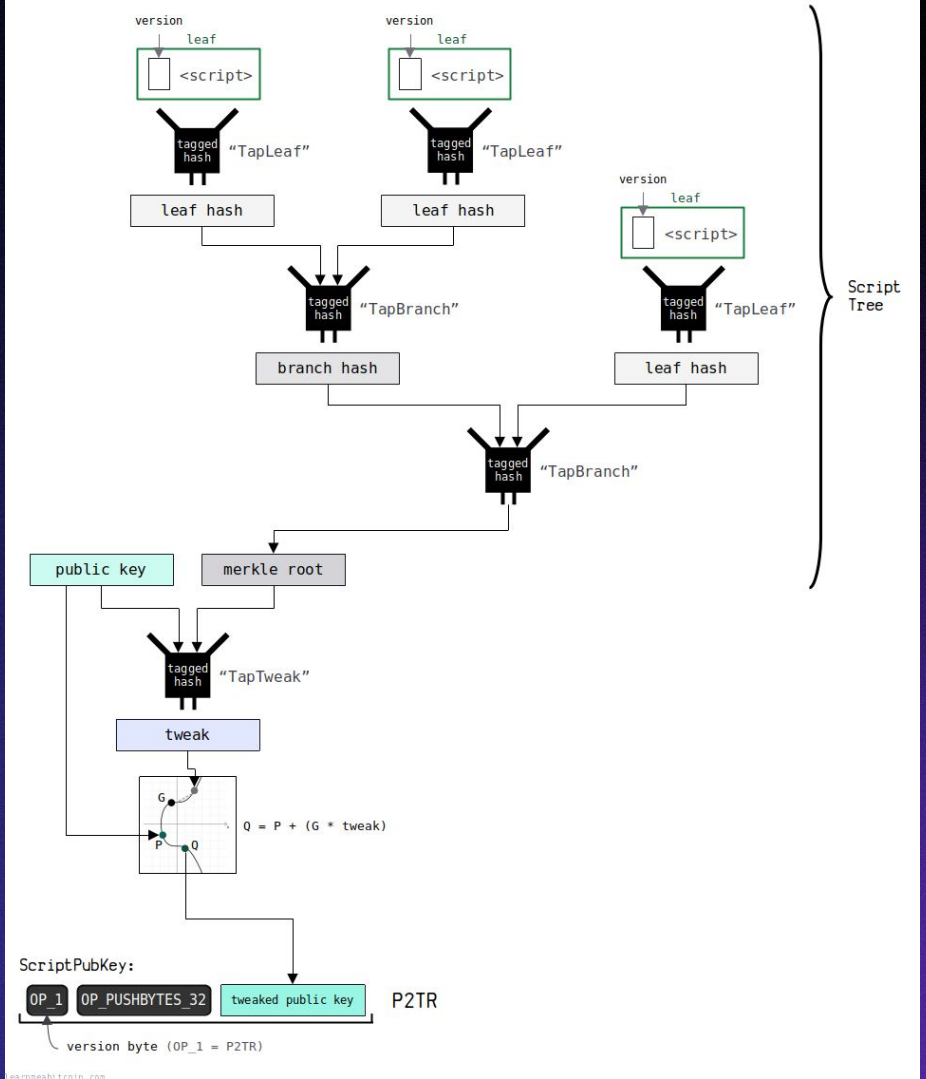
- Tweak
- Tweaked Public Key

Step 3: creating the ScriptPubKey

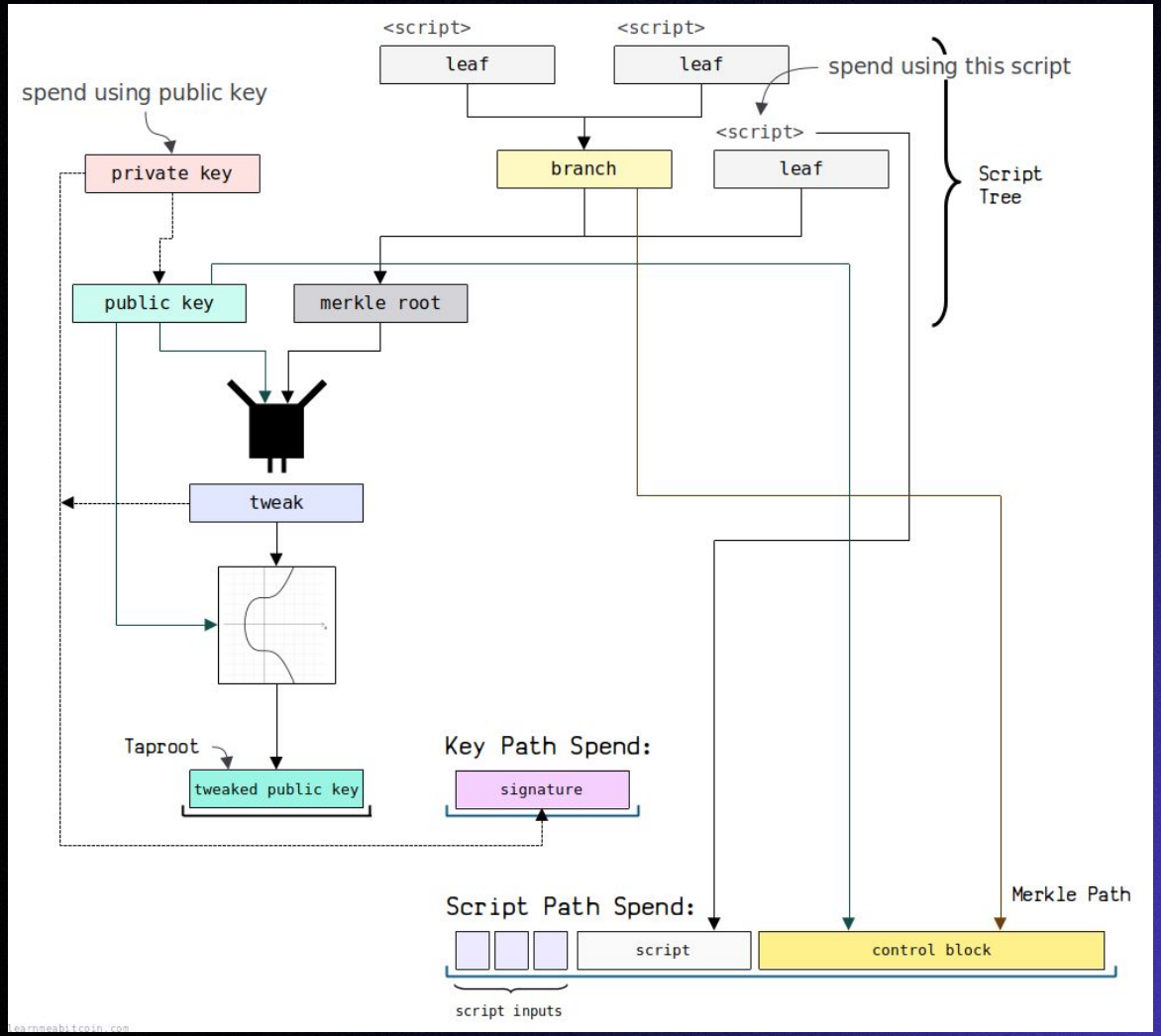
Finally, we place this tweaked public key into a locking script to create a P2TR output:

OP_1	OP_PUSHBYTES_32	<32-byte tweaked public key>

So a P2TR locking script is just a data push of the tweaked public key, prepended with an OP_1 opcode to indicate that it's a P2TR.



Overview of spending paths



Key Path Spend :

This is where you provide a signature for the public key used to create the locking script. This works in a similar way to a simple P2WPKH.

Script Path Spend:

This is where you use one of multiple possible custom scripts to unlock the output. This is like using P2WSH, but you have multiple custom scripts to choose from.

Key-Path Spend

A P2TR key path spend is where you unlock the output via the default public key.

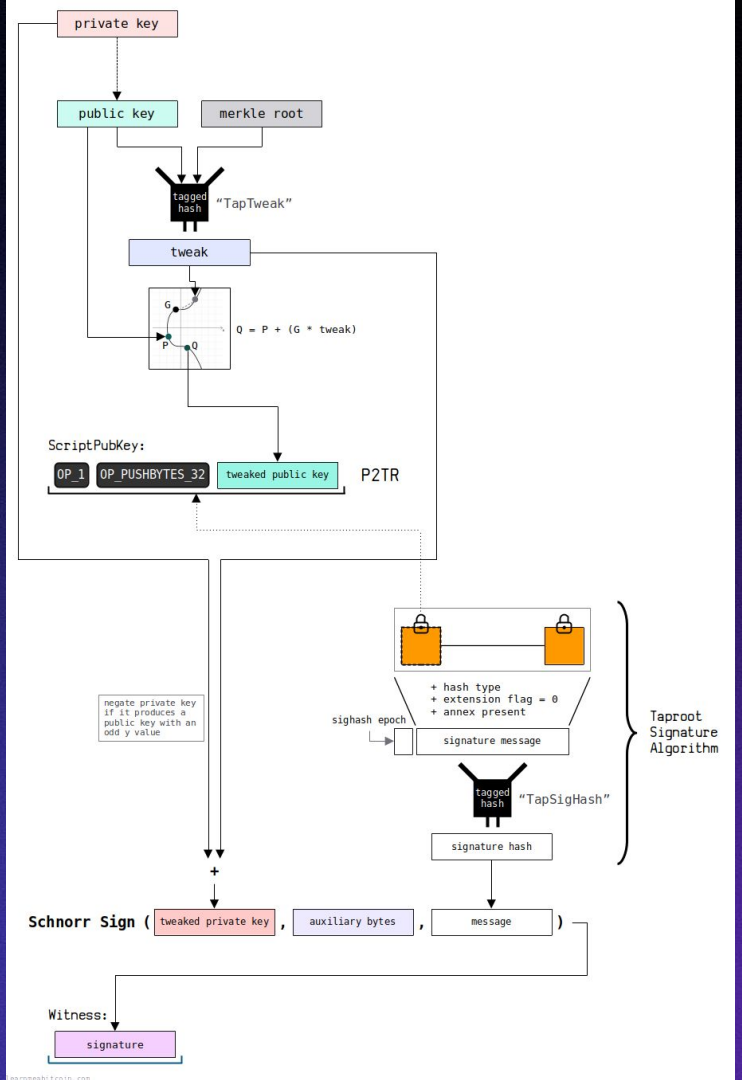
This spend is fairly straightforward, as you only need to provide a signature that corresponds to the tweaked public key in the ScriptPubKey.

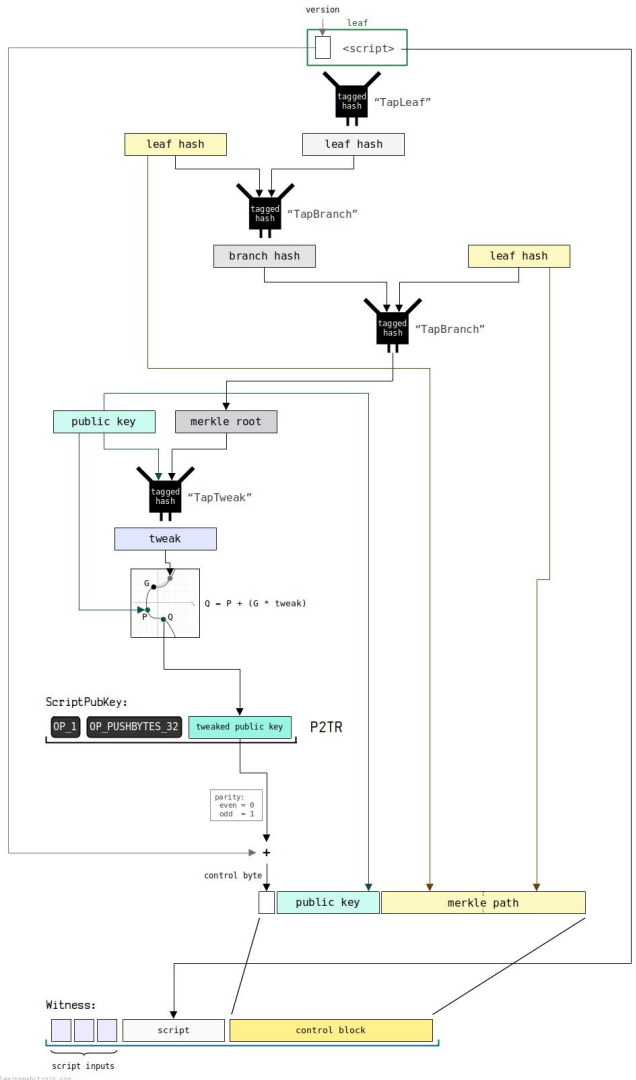
A key path spend is performed in 3 steps:

- Tweaked Private Key
  - The tweaked private key is calculated by simply adding the tweak to the original private key.
- Signature Hash
  - To create a signature from this tweaked private key, you first need to construct a message to be signed.
  - This message is called the signature hash, which is a hash of the transaction data constructed in a specific way.
- Signature
  - The signature for a key path spend is created by signing the signature hash from the previous step using the tweaked private key.

Witness:

Finally, after creating the signature you just need to construct a witness field to unlock the P2TR output.





Script-Path Spend

A script path spend requires 3 things:

1. A leaf script from the script tree.
2. The inputs required to unlock that leaf script.
3. A merkle path proving that leaf script was part of the script tree.

Each of these 3 components are then placed into the witness.

The part we're interested in is the "Control Block" of the witness. This control block contains the merkle path that proves the previous leaf script was part of the original script tree. It also contains the original public key, which is required to verify that the public key and merkle root can be combined to match the tweaked public key in the ScriptPubKey.

The complete control block is split into 3 sections:

- Control Byte
  - This first byte of the control block is the control byte. This encodes two pieces of data:
    - The leaf version of your chosen leaf script.
    - A bit indicating the parity of the y-coordinate of the full tweaked public key point.
- Public Key
  - The next 32 bytes of the control block contains the original public key.
  - This needs to be revealed so that the tweaked public key can be calculated from it.
- Merkle Path
  - The remaining data in the control block is the merkle path.

Why was all of this important??

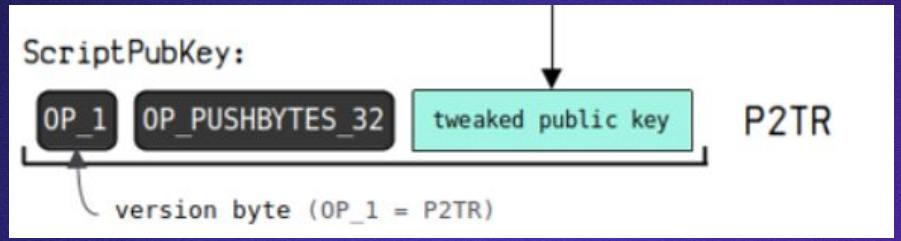
This is supposed to be a presentation on Pay-To-Merkle-Root, so what's all of this then?

This was all important to know moving forward because it's needed to understand the problem that Pay-To-Merkle-Root is trying to solve.

And what is this Problem that Pay-To-Merkle-Root has set out to solve?
It's the Key Path Problem.

Remember that to support using a private key to spent the transaction we have to include the tweaked public key in the ScriptPubKey which we publish on chain!

This can be dangerous!!



The Key Path Problem

- P2TR has to publish the tweaked public key on-chain permanently to support key path spend
- Older formats: key hidden behind hash until spend
- Taproot: long-exposure risk becomes the default

Long-Exposure Risk

- Public key on-chain until spent
- Attacker has unlimited time
- Dormant coins most vulnerable

Short-Exposure Risk

- Mempool window: ~10 minutes
- Attacker must crack key before confirmation
- Pre-Taproot: only risk with good hygiene

BIP 360: Pay-to-Merkle-Root (P2MR)

Take Taproot and Remove the Key-Path Spend Entirely

- Address commits only to Merkle root of script tree
- No internal public key baked into address
- Every spend goes through the script path
- Public key appears only inside the revealed script branch

Technical details:

- SegWit v2
- Full Tapscript compatibility
- Soft fork, backwards compatible
- 32-byte outputs, reuses Taproot code

This is the entire proposal. It removes the keypath spend from Taproot. That’s it.

What P2MR Solves

What P2MR Does

- Eliminates gratuitous public key exposure from Taproot's key-path
- Provides a migration target, new address type with no key on-chain
- Preserves scripting capabilities for Lightning, BitVM, Ark
- Creates the foundation for post-quantum signature schemes

What P2MR Does NOT Do

- Does not protect already-exposed public keys
- Does not introduce quantum-resistant cryptography
- Short-exposure risk (mempool window) remains
- Does not address dormant coins in vulnerable addresses

Why Not Add Quantum-Resistant Signatures Directly?

The tradeoffs of post quantum

- Post-quantum signatures are huge:
  - Dilithium: ~ 2–3 KB per signature
  - Current (ECDSA/Schnorr): ~ 64 bytes
- No consensus on which algorithm
  - ML-DSA, SLH-DSA, FN-DSA
- Block size and fee pressure debates
- Governance: agreeing on one (or many) is hard
- Timeline: consensus takes years; threat may arrive sooner

The Landscape of Solutions

Post-Quantum Signatures

- Future soft forks into P2MR script trees
- Dilithium, SPHINCS+ candidates
- Would close short-exposure vulnerability

“Hourglass” Approach

- Spending restrictions on exposed addresses
- Give owners a migration window
- Politically explosive, touches “valid crypto = ownership” argument

OP_CAT + Lamport Signatures

- BIP-347: reintroduce OP CAT
- Hash-based, quantum-resistant
- Bitcoin had this path from day one, Satoshi removed it in 2010
- Downside: very large, single-use

The Migration Path

01

Soft Fork, P2MR becomes valid

This will cause no friction as it is compatible with P2TR

02

Wallet software updated to support P2MR addresses

Allow time for adoption of the protocol

03

Users migrate funds: send BTC to new P2MR addresses

Allow time for users to change over to new wallets

04

Future soft forks enable PQ signatures inside P2MR script trees

Allow for the adoption of post quantum signatures in this new scheme

Closing

BIP 360 is a pragmatic, conservative first step and nothing more.

- Does not solve the quantum problem
- Creates the conditions under which it can be solved
- Removes Taproot's quantum-vulnerable key-path
- Preserves everything the scaling ecosystem depends on
- Foundation for future post-quantum upgrades

Thank you!

QUESTIONS?